

האוניברסיטה העברית

ביה"ס להנדסה ולמדעי המחשב

בחינה בקורס מבוא למדעי המחשב 67101

מועד א' – סמסטר א' – תשע"ה – 2014-2015

תאריך: 10.2.2015

ד"ר אביב זוהר, פרופ' יובל רבני.

משך הבחינה: שלוש שעות

הבחירות:

1. הבחינה עם חומר סגור. אין להביא חומר עזר כלשהו לבחינה (כולל ספרים, מחברות, דפים ומכשירים אלקטרוניים). יש להיעזר רק בדף המצורף לבחינה, ובו רשימת פונקציות ומתודות שניתן להשתמש בהן.
2. יש לענות על 3 מתוך 4 השאלות הבאות. משקל כל השאלות זהה (33 נק'), נקודה אחת נוספת מוענקת אוטומטית, כך שהציון המקסימלי הוא 100.
3. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת בפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור. אין לענות על גבי טופס הבחינה - הוא לא נמסר בסיום הבחינה ולא יבדק.
4. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
5. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
6. מותר להשתמש בכל הפונקציות והמתודות הכתובות בדף המצורף לבחינה, וכן בכל הפונקציות שהן **built-in** בפייתון, בתנאי שהן לא פותרות ישירות את השאלה.
7. בפייתון ההזחה (אינדנטציה) היא קריטית להבנת הקוד – הקפידו על הזחה ברורה ונכונה!
8. יש למחוק טיוטות באופן ברור. הבדיקה לא תתייחס לטיוטות כלל.
9. יש להשאיר שוליים.
10. אסור לכתוב בעט אדום.
11. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
12. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית). אין צורך לכתוב docstring.
13. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
14. אין להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להשתמש ברשימה בת שלשה תאים פעם אחת).
15. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. ניתן להניח כי מבני הנתונים מכילים רק ערכים תקינים ואינם None, אלא אם מובהר בשאלה אחרת.
16. בפתרון השאלה אתם רשאים לבצע שימוש בפונקציות ומתודות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בפונקציות או מתודות משאלות אחרות).

בהצלחה!

שאלה 1 (33 נקודות)

יש לענות על הסעיפים הבאים על פי הקוד הנתון:

```
a = [0,1,2,3,4,5,6,7,8,9]
```

א. (5 נקודות) מה יודפס ב-3 ההדפסות הבאות?

```
print(a[8:2:2])
print(a[:2:-2])
print(a[8: :-2])
```

```
def rev(s):
    return s[::-1]
```

ב. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(rev('beauty'))
```

```
def f(s):
    if len(s) < 2:
        return s
    else:
        s2 = s[0:2]

        if s[0] > s[1]:
            s2 = rev(s2)
        return s2[0] + f(s2[1]+s[2:])
```

ג. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(f('beauty'))
```

ד. (3 נקודות) מה צריך לשנות בשורה האחרונה של הפונקציה f כך שהיא תעבוד על קלט s שהוא list של ערכי int?

עבור ההמשך, הניחו שהקוד של f נותר כפי שהוא למעלה, ללא השינוי שהכנסתם בסעיף ד.

```
def foo(s, n):
    s1 = s
    for i in range(n):
        s1 = f(s1)
    return s1
```

ה. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(foo('beauty', 2))
```

ו. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(foo('beauty', len('beauty')))
```

המשך השאלה בעמוד הבא.

```
def fooy(s, i, b):
    if i >= len(s)//2:
        return '' # empty string
    else:
        s2 = s[i] + s[i+len(s)//2]
        if b:
            s2 = rev(s2)
        return s2 + fooy(s, i+1, b)
```

ז. (4 נקודות) מה יודפס בשתי ההדפסות הבאות?

```
print(fooy('beauty', 0, False))
print(fooy('beauty', 0, True))
```

ח. (3 נקודות) אם נשנה את השורה האחרונה ב-fooy ל-
 return s2 + fooy(s, i+1, not b) מה תדפיס הקריאה הבאה:

```
print(fooy('beauty', 0, False))
```

ט. עבור ההמשך, הניחו שהקוד של fooy נותר כפי שהוא למעלה, ללא השינוי בסעיף ח'.

י. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(foo(fooy('beauty', 0, False), 5))
```

יא. (3 נקודות) מה יודפס בהדפסה הבאה?

```
print(fooy(foo('beauty', 5), 0, True))
```

שאלה 2 (33 נקודות)

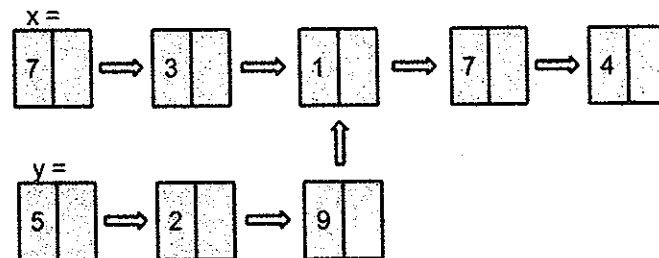
נתונה המחלקה Node המייצגת איבר ברשימה מקושרת (סופית).

```
class Node:
    def __init__(self, data, next_node=None):

        self.data = data
        self.next = next_node
```

נתונים שני אובייקטים מטיפוס Node, שנסמן ב-X ו-Y, שהם ראשים של שתי רשימות מקושרות.

הגדרה: נאמר כי שתי הרשימות מתלכדות אם יש להן איברים משותפים בסוף הרשימות. לדוגמה, הרשימות באיור מספר 1 מתלכדות שכן שני האיברים האחרונים שייכים גם ל-X ול-Y. שימו לב כי בשתי רשימות מתלכדות, החל מהאיבר הראשון המשותף, כל האיברים הבאים גם הם משותפים. להבהרה: אם node_x מ-X ו-node_y מ-Y משותפים, אז node_x is node_y.



איור מספר 1

שימו לב:

- בכל סעיף, חתימת הפונקציה הנדרשת מופיעה בסוף הסעיף.
- בכל הסעיפים ניתן לכתוב פונקציה שסיבוכיות הזמן שלה ליניארית (כלומר $O(n)$) באורך הרשימה המקושרת (או הרשימות). פתרונות פחות יעילים יובילו להורדה של עד 7 נקודות מכלל השאלה.
- בכל סעיף, מותר להשתמש בפונקציות מהסעיפים הקודמים, גם אם לא הצלחתם לכתוב אותן. שכפול קוד יגרור הורדת נקודות. עשו שימוש בפונקציות מסעיפים קודמים אם ניתן לעשות זאת ביעילות.
- אין צורך להגדיר מחלקה של רשימה מקושרת בשאלה זו.
- בשאלה זו, תתקבלנה תשובות החוסכות בקוד במחיר של הפסד קטן בסיבוכיות - כל עוד סיבוכיות הפונקציה נשארת ליניארית.

א. (3 נקודות) ממשו את הפונקציה len_list המקבלת מצביע head לראש רשימה מקושרת (אובייקט מטיפוס Node) ומחזירה את אורך הרשימה. אם הרשימה ריקה עליכם להחזיר 0.

```
def len_list(head):
```

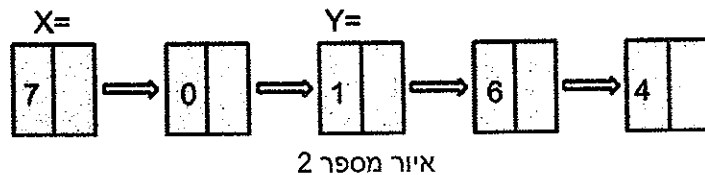
ב. (6 נקודות) ממשו את הפונקציה node_i המקבלת מצביע head לראש של רשימה מקושרת (אובייקט מטיפוס Node) ואינדקס idx (ערך מטיפוס int). אם האינדקס אי-שלילי (כלומר גדול או שווה ל-0), הפונקציה מחזירה מצביע לאיבר ה-idx ברשימה (אובייקט מטיפוס Node). במקרה זה האינדקס של האיבר הראשון ברשימה מוגדר להיות 0. אם האינדקס הוא מספר שלילי, הפונקציה מחזירה את האיבר ה-(-idx) מסוף הרשימה. במקרה זה, האינדקס של האיבר האחרון ברשימה מוגדר להיות -1. אם הרשימה ריקה או שאין ברשימה איבר שמקומו מתאים ל-idx, עליכם להחזיר None.

```
def node_i(head, idx):
```

המשך השאלה בעמוד הבא.

ג. (7 נקודות) הגדרה: רשימה מקושרת Y היא זנב של רשימה מקושרת X אם כל האיברים של Y מוכלים ב-X, כך שאיברי הרשימה Y הם האיברים האחרונים ב-X.

באיור מספר 2 המצורף Y היא זנב של X.



הערה: שימו לב כי רשימה ריקה (None) אינה זנב של רשימה אחרת.

ממשו את הפונקציה `is_tail` שמקבלת שני מצביעים `head_x` ו-`head_y` לראשים של שתי רשימות מקושרות (אובייקטים מטיפוס `Node`) ובודקת אם הרשימה השנייה `head_y` היא זנב של הרשימה הראשונה `head_x`. על הפונקציה להחזיר ערך בוליאני (`True` אם `head_y` היא זנב של `head_x` ו-`False` אחרת).

```
def is_tail(head_x, head_y):
```

ד. (9 נקודות) ממשו את הפונקציה `find_joint_node` שמקבלת שני מצביעים `head_x` ו-`head_y` לראשים של שתי רשימות מקושרות (אובייקטים מטיפוס `Node`) ובודקת אם הרשימות מתלכדות (הזכרו בהגדרה של רשימות מתלכדות בתחילת השאלה). אם הרשימות מתלכדות על הפונקציה להחזיר מצביע לאיבר הראשון המשותף לשתי הרשימות (אובייקט מטיפוס `Node`), אחרת עליה להחזיר `None`.

רמז: שרטטו שתי רשימות מתלכדות באורכים שונים במקביל (כאשר האיברים הזהים מופיעים פעמיים), מיושרות לפי האיבר האחרון (שהוא בהכרח זהה, אם הן מתלכדות). סמנו את האיברים הזהים בשתייהן.

```
def find_joint_node(head_x, head_y):
```

ה. (8 נקודות) ממשו פונקציה `combine` שמקבלת שני מצביעים `head_x` ו-`head_y` לראשים של שתי רשימות מקושרות (אובייקטים מטיפוס `Node`) ומחזירה מצביע לרשימה מקושרת חדשה (אובייקט מטיפוס `Node`), שמכילה את האיברים של שתייהן. אסור לשנות את רשימות הקלט. **שימו לב**: ברשימת הפלט צריכים להופיע איברים חדשים שמחזיקים העתק של שדה ה-`data` של האיברים המקוריים ברשימות הקלט. סדר האיברים ברשימת הפלט אינו חשוב, אבל יש להקפיד שעבור כל איבר ברשימות הקלט יופיע איבר ברשימת הפלט עם העתק של ה-`data` שלו. אם איבר מופיע בשתי רשימות הקלט (כי הן מתלכדות) אז צריך להופיע רק העתק אחד שלו ברשימת הפלט.

```
def combine(head_x, head_y):
```

שאלה 3 (33 נקודות)

בשאלה זו צריך לבנות מחלקה המייצגת רשימה של מספרי `int` חיוביים, כל אחד בן 9 ספרות לכל היותר. כל מספר `int` כזה מיוצג על פי סדרת הספרות שלו משמאל לימין. הייצוג הפנימי במחלקה חייב להיות רשימה אחת בלבד (`list`) המכילה ערכי `int` בין 0 ל-9 בלבד. נציג שתי דרכים לעשות זאת. עליכם לממש את שתיהן בצורה של מחלקות נפרדות, על פי הפונקציות הנדרשות בסעיפים ב' ו-ג' של השאלה.

דרך א' - המחלקה `FixedWidthList`:
במחלקה זו כל איבר ברשימה המקורית מיוצג על ידי אותו מספר של תאים, כלומר 9 תאים. כדי לייצג מספרים קצרים יותר, מוסיפים אפסים משמאל.

לדוגמה, רשימת המספרים: `[4, 123456789]` תיוצג בתוך המחלקה הזו ע"י:
`[0, 0, 0, 0, 0, 0, 0, 0, 4, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

דרך ב' - המחלקה `VariableWidthIntList`:
במחלקה זו כל איבר ברשימה המקורית מיוצג על ידי תא אחד שמחזיק את מספר הספרות הדרוש ואחריו תאים שמחזיקים את הספרות באיבר לפי הסדר משמאל לימין.

לדוגמה, רשימת המספרים: `[4, 123456789]` תיוצג בתוך המחלקה הזו ע"י:
`[1, 4, 9, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

בכל הסעיפים ניתן להניח שהקלט חוקי לפי הדרישות בסעיף והדרישות לעיל.

א. (6 נקודות) כתבו איטרטור שמאותחל על ידי מספר `int` חיובי ומחזיר את סדרת הספרות שלו משמאל לימין. לדוגמה, אם האיטרטור מאותחל על ידי המספר 432, אז באיטרציה הראשונה הוא יחזיר את המספר 4, באיטרציה השנייה 3, באיטרציה השלישית 2, ואח"כ הוא יסיים את פעולתו. במימוש ניתן להשתמש בגנרטור או להגדיר מחלקה נוספת של האיטרטור שיוחזר, כרצונכם. הערה: אין לפתור סעיף זה ע"י המרה למחרוזת, אך ניתן לעשות שימוש בפעולות חשבוניות.

ב. (12 נקודות) ממשו במחלקה `FixedWidthInt` את 3 המתודות הבאות (המתודה השנייה והמתודה השלישית מופיעות בעמוד הבא):

```
1. def __init__(self, list_of_ints):
```

הפרמטר `list_of_ints` הוא אובייקט `iterable` כלשהו שהאיטרטור שלו מחזיר סדרה של ערכי `int` חיוביים, כל אחד בן 9 ספרות לכל היותר. באובייקט הנוצר על ידי ה-`__init__` חייב להיות שדה יחיד שמחזיק `list` של ערכי `int` שכל אחד מהם הוא בין 0 ל-9.

המשך השאלה בעמוד הבא.

2. `def __iter__(self):`

המתודה מייצרת ומחזירה איטרטור שמחזיר את סדרת ערכי ה-`int` המקורית (כלומר, את סדרת ערכי `list_of_ints` ששימשה לייצור האובייקט `self`). לדוגמה, אם האובייקט שומר את הרשימה `[0, 0, 0, 0, 0, 0, 0, 1, 2, 0, 0, 0, 0, 0, 0, 2, 3, 4, 0, 0, 0, 0, 3, 4, 5, 6]` אז האיטרטור שנוצר צריך להחזיר באיטרציה הראשונה את המספר 12, באיטרציה השנייה את המספר 234, באיטרציה השלישית את המספר 3456, ואח"כ הוא יסיים את פעולתו. במימוש ניתן להשתמש בגנרטור או להגדיר מחלקה נוספת של האיטרטור שיוחזר, כרצונכם.

3. `def __getitem__(self, index):`

מחזירה את האיבר במקום `index` בסדרת ערכי ה-`int` המקורית (כלומר בסדרת ערכי `list_of_ints` ששימשה לייצור האובייקט `self`). שימו לב: ערך `index` של האיבר הראשון הוא 0. אם `index` חורג מהתחום האפשרי, יש להחזיר 1-.

ג. (12 נקודות) ממשו במחלקה `VariableWidthInt` את אותן 3 מתודות כמו בסעיף ב':

1. `def __init__(self, list_of_ints):`

הפרמטר `list_of_ints` הוא אובייקט `iterable` כלשהו שהאיטרטור שלו מחזיר סדרה של ערכי `int` חיוביים, כל אחד בן 9 ספרות לכל היותר. באובייקט הנוצר על ידי ה-`__init__` חייב להיות שדה יחיד שמחזיק `list` של ערכי `int` שכל אחד מהם הוא בין 0 ל-9.

2. `def __iter__(self):`

המתודה מייצרת ומחזירה איטרטור שמחזיר את סדרת ערכי ה-`int` המקורית (כלומר, את סדרת ערכי `list_of_ints` ששימשה לייצור האובייקט `self`). לדוגמה, אם האובייקט שומר את הרשימה `[2, 1, 2, 3, 2, 3, 4, 4, 3, 4, 5, 6]` אז האיטרטור שנוצר צריך להחזיר באיטרציה הראשונה את המספר 12, באיטרציה השנייה את המספר 234, באיטרציה השלישית את המספר 3456, ואח"כ הוא יסיים את פעולתו. במימוש ניתן להשתמש בגנרטור או להגדיר מחלקה נוספת של האיטרטור שיוחזר, כרצונכם.

3. `def __getitem__(self, index):`

מחזירה את האיבר במקום `index` בסדרת ערכי ה-`int` המקורית (כלומר בסדרת ערכי `list_of_ints` ששימשה לייצור האובייקט `self`). שימו לב: ערך `index` של האיבר הראשון הוא 0. אם `index` חורג מהתחום האפשרי, יש להחזיר 1-.

ד. (3 נקודות) מה היתרונות והחסרונות של כל מחלקה? התמקדו בניתוח המקום הנדרש כדי לייצג אותו אובייקט בשתי השיטות ובזמן הנדרש לחישוב ב-`__getitem__`. מספיק לציין מהי סיבוכיות המקום והזמן עבור כל אחת מהמחלקות, אין צורך לנמק מדוע זו הסיבוכיות.

שאלה 4 (33 נקודות)

נשתמש ב-tuple של פייתון (a,b) על מנת לייצג קטע סגור על הישר הממשי, כלומר את קבוצת המספרים הממשיים x עבורם $a \leq x \leq b$. בשאלה ניתן להניח שבכל הקטעים שמקבלים כקלט מתקיים: $a \leq b$, וגם ש- a ו- b הם ערכי int.

בכל סעיף, חתימת הפונקציה הנדרשת מופיעה בסוף הסעיף.

א. (5 נקודות) שני קטעים נחתכים אם קיימת לפחות נקודה אחת המוכלת בשניהם. כתבו פונקציה `intersect` המקבלת כקלט שני קטעים `segment1`, `segment2` (הנתונים כ-tuple לפי ההסבר לעיל) ובודקת אם הם נחתכים. הפונקציה מחזירה ערך בוליאני - True אם הקטעים נחתכים, ו-False אחרת. לדוגמא: שלושת הזוגות של הקטעים בשורות להלן נחתכים:

(0,5) (2,7)
(2,7) (1,8)
(-3,1) (1,2)

אולם, שני הזוגות של הקטעים בשורות להלן לא נחתכים:

(0,5) (6,7)
(0,2) (-2,-1)

```
def intersect(segment1, segment2):
```

ב. (6 נקודות) ממשו פונקציה `has_intersection` שמקבלת רשימה `segment_list` של קטעים (list שבו כל קטע מיוצג ע"י tuple כ"ל), ומחזירה True אם קיימים שני קטעים ברשימה שנחתכים, ו-False אחרת. בסעיף זה אסור למיין את רשימת הקלט.

```
def has_intersection(segment_list):
```

בסעיפים הבאים מותר למיין את הרשימה. אין צורך לכתוב את המיין. קריאה ל-`segment_list.sort()` ממיינת את הרשימה לפי הקואורדינטה הראשונה של כל tuple ברשימה (כלומר לפי ערכי a בקטע) בסדר עולה (ובמקרה של שיוויון לפי הקואורדינטה השנייה).

ג. (8 נקודות) כתבו מימוש יעיל יותר לפונקציה מהסעיף הקודם.

```
def more_efficient_has_intersection(segment_list):
```

ד. (3 נקודות) ציינו את סיבוכיות זמן הריצה של הפונקציה `has_intersection` שמימשתם בסעיף ב' ואת סיבוכיות זמן הריצה של `more_efficient_has_intersection` שמימשתם בסעיף ג'. אפשר להניח שגישה לאיבר של list או tuple לוקחת זמן קבוע, כלומר $O(1)$ פעולות, ושהמתודה `sort` עובדת על רשימה באורך n בזמן $O(n \log n)$. מספיק לציין מהי סיבוכיות זמן הריצה, אין הכרח לנמק את תשובתכם (נימוק יכול לעזור חלקית אם החישוב שלכם שגוי אבל הרעיון נכון).

ה. (11 נקודות) נאמר שרשימה של קטעים `segment_list` מכסה קטע אחר `segment` אם כל נקודה בקטע `segment` מופיעה בלפחות אחד מהקטעים ברשימה `segment_list`. כתבו פונקציה `covers` שמקבלת רשימת קטעים `segment_list` (נתונה כ-list של tuples כ"ל) וקטע `segment` (נתון כ-tuple כ"ל), ומחזירה True אם הרשימה `segment_list` מכסה את `segment`, ו-False אחרת.

דוגמא: הקטעים $(0,1)$, $(1,3)$, $(2,5)$ מכסים את הקטע $(0,4)$. אך הקטעים $(0,1)$, $(2,3)$, $(2,5)$ לא מכסים את הקטע $(0,4)$ (למשל, כי הנקודה 1.5 לא נכללת באיחוד של הקטעים ברשימה, אבל כן נכללת בקטע $(0,4)$).

```
def covers(segment_list, segment):
```

רשימת פונקציות ומתודות מהספרייה של פייתון:

ברשימה זו המילים: *bool, float, int, number, object, iterable, iterator, dict, str, tuple, list* מייצגים אובייקט כלשהו מטיפוס כזה.

הסימון *x* מייצג פרמטר שיכול להיות מטיפוסים שונים, אבל לא אובייקט כללי.

הסימונים *string, copy, value* מייצגים פלט שטיפוסו תלוי בפרמטרים שניתנים לפונקציה.

הסימון *func* מייצג פרמטר שהוא פונקציה (מספר הפרמטרים שלה והערך שהיא מחזירה תלוי בהקשר).

המרות ויצירת אובייקטים:

```
bool = bool(x)  
float = float(x)  
int = int(x)  
list = list(iterable)  
str = str(object)  
tuple = tuple(iterable)  
copy = copy.deepcopy(object)
```

convert a value to a boolean
convert a number or a string to float
convert a number or a string to int
create a list of the items of **iter**(*iterable*)
convert to a string, using *object.__str__*()
create a tuple of the items of **iter**(*iterable*)
create a deep copy of *object*

פונקציות מתמטיות:

```
number = abs(number)  
value = max(iterable)  
value = max(x1, x2, ...)  
value = min(iterable)  
value = min(x1, x2, ...)  
int = round(number)  
value = sum(iterable)
```

return the absolute value of *number*
return the maximum value in **iter**(*iterable*)
same, given an explicit list of values
return the minimum value in **iter**(*iterable*)
same, given an explicit list of values
round the number
return the sum of the items of **iter**(*iterable*)

איטרטורים:

```
iterator = iter(iterable)  
value = next(iterator)  
list = sorted(iterable)
```

create the iterator of *iterable*
return the next value from an iterator
return a sorted list of the items of **iter**(*iterable*)

מתודות של *list*:

```
list.append(x)  
value = list.pop()  
value = list.pop(int)  
list.sort()  
list.insert(int, x)
```

add *x* to the end of *list*
return and remove the last item in *list*
return and remove the item in *list[int]*
sort a list of comparable objects in place
insert an item *x* at a given position *int*

מתודות של *str*:

```
string = str.join(iterable)  
  
list = str.split()  
list = str.split(sep)
```

concatenate the items of **iter**(*iterable*), separated by *str*
split *str* into words separated by white space
split *str* into words separated by the input string *sep*

